
pyCATCH

Release 1.0.0

Stephan G. Heinemann

Oct 01, 2026

CONTENTS:

1	Getting started with pyCATCH	3
1.1	Reading the documentation	3
1.2	Installing pyCATCH	3
1.3	Initializing pyCATCH	4
1.4	Downloading data	4
1.5	Loading data	4
1.6	Calibrating data	4
1.7	Quicklook at the data	5
1.8	Selecting coronal hole seed point	5
1.9	Setting a threshold	5
1.10	Extracting the coronal hole	5
1.11	Calculating coronal hole properties	6
1.12	Plotting and saving coronal hole extractions	6
1.13	Saving and loading pyCATCH session	6
1.14	Example	6
2	pyCATCH	7
2.1	pycatch.pycatch	7
2.2	pycatch.utils	16
3	Changelog	33
3.1	1.0.0 (October 1, 2026)	33
3.2	0.2.1 (Novemeber 9, 2023)	33
3.3	0.2.0 (September 2023)	34
3.4	0.1.0 (September 2023)	34
4	Acknowledging or Citing pyCATCH	35
4.1	DISCLAIMER	35
5	References	37
	Python Module Index	39
	Index	41



Welcome to pyCATCH: the python implementation of the Collection of Analysis Tools for Coronal Holes (CATCH; Heinemann et al. 2019)

CATCH was originally implemented in SSW IDL (See http://lmsal.com/solarsoft/ssw_install.html and http://www.lmsal.com/solarsoft/ssw_packages_info.html). It is also available on GitHub (<https://github.com/sgheinemann/CATCH>)

CATCH and pyCATCH were created in order to collect and structure coronal hole identification, extraction and analysis in a handy and fast way without the disadvantages of automatic algorithms. It enables the user to download and process EUV filtergrams (193/195 Å) and line-of-sight (LoS) magnetograms. It is able to handle data from different spacecraft missions covering the interval from 1996 until now. These include the Solar Dynamics Observatory, the Solar Terrestrial Relations Observatory and the Solar and Heliospheric Observatory.

The user can perform coronal hole boundary detection, extraction and analysis using a manually adjustable intensity threshold. Additionally the user can analyze the underlying photospheric magnetic field.

If you have any comments, suggestions or need help. Please contact the author via E-mail (stephan.heinemann@hmail.at) or leave a request on Github (<https://github.com/sgheinemann/CATCH> or <https://github.com/sgheinemann/pycatch> respectively).

You can read the documentation by opening the

doc/_build/index.html

file.

GETTING STARTED WITH PYCATCH

This section provides a quick introduction to using pyCATCH.

pyCATCH essentially consists only of the class object pycatch that included inbuilt methods for detection and extraction of coronal holes.

It uses a lot of sunpy <<https://sunpy.org/>> functionalities, thus for advanced usage of pyCATCH, it is suggested to make yourself familiar with it.

1.1 Reading the documentation

You can read the documentation by opening the

```
doc/_build/html/index.html
```

file.

1.2 Installing pyCATCH

Current pyCATCH version: Download or pull pyCATCH repository from GitHub

```
https://github.com/sgheinemann/pycatch
```

Navigate to the directory and install pyCATCH using pip

```
pip install .
```

pyCATCH uses the following packages:

- python >= 3.14
- numpy ~= 2.5.3
- astropy ~= 8.0.1
- sunpy ~= 8.0.0
- aiapy ~= 0.12.0
- opencv-python ~= 5.0.0
- matplotlib ~= 3.11.1
- reproject ~= 0.21.0
- scipy ~= 1.18.1

- numexpr ~= 2.14.2
- joblib ~= 1.6.0
- jupyterlab ~= 4.6.4
- ipympl ~= 0.10.0

1.3 Initializing pyCATCH

Import the pycatch class with

```
from pycatch.pycatch import pycatch
```

then you can initialize the class

```
ch = pycatch()
```

1.4 Downloading data

EUV data can be downloaded with

```
ch.download('DATE', email='example@pycatch.com')
```

a JSOC registered email is required for the default download routine. Magnetograms can be automatically downloaded, if a EUV map is already loaded, with

```
ch.download_magmagnetogram(email='example@pycatch.com')
```

1.5 Loading data

EUV images and magnetograms can be loaded into the pyCATCH class by

```
ch.load(file='PATH')  
ch.load(file='PATH', mag = True )
```

If data was downloaded with the same instance of the pyCATCH class, the path to the downloaded data is stored in the class and the load method can be called without providing the path.

1.6 Calibrating data

Prepare the data for coronal hole extraction

```
ch.calibration()  
ch.calibration_mag()
```

The data can be rebinned and cutout if desired

```
ch.rebin(ndim=(nx,ny))  
ch.cutout(top=(x_max,y_max), bot=(x_min,y_min))
```

1.7 Quicklook at the data

Data can be quickly visualized with

```
ch.show()
ch.show(mag=True)
```

1.8 Selecting coronal hole seed point

Select a seed point from an interactive window from where the coronal hole will be grown

```
ch.select(hint=True)
```

Flag hint to highlight dark regions (possible but not necessarily all coronal hole).

1.9 Setting a threshold

pyCATCH features four different options to select a threshold.

- Set the threshold manually:

```
ch.set_threshold(Threshold)
```

- Use the threshold derived from CATCH statistics (Heinemann et al. 2019):

```
ch.suggest_threshold()
```

It is advised to use this option to get a starting suggestion and then adjust the threshold as needed.

- Select the threshold from solar disk intensity distribution:

```
ch.threshold_from_hist()
```

- Calculate the coronal hole area and uncertainty as function of intensity and select the boundary to be where the uncertainty is lowest:

```
ch.calculate_curves()
ch.threshold_from_curves()
```

Warning

This is an advanced option and can be slow with high resolution.

1.10 Extracting the coronal hole

The coronal hole can be extracted from the selected seed point and the set threshold

```
ch.extract_ch()
```

1.11 Calculating coronal hole properties

The coronal hole's morphological properties can be calculated with

```
ch.calculate_properties()
```

and the magnetic properties can be calculated with

```
ch.calculate_properties(mag=True)
```

1.12 Plotting and saving coronal hole extractions

The intensity map with the coronal hole boundary and uncertainties overlaid can be displayed with

```
ch.plot_map()
```

Set the 'mag' flag to display the magnetogram instead

```
ch.plot_map(mag=True)
```

Set the 'save' flag to save the images to PDF

```
ch.plot_map(save=True)  
ch.plot_map(mag=True, save=True)
```

The properties of the extracted coronal hole can be saved in a text file with

```
ch.print_properties()
```

And the extracted coronal hole 'binary' map (or any other map) can be saved as a fits file with

```
ch.save_map()
```

1.13 Saving and loading pyCATCH session

A pyCATCH object can be save to a pickle file

```
ch.save()
```

and restored with

```
ch_loaded = pycatch(restore='PATH')
```

1.14 Example

Example python scripts can be found in the directory

```
examples/
```

PYCATCH

2.1 pycatch.pycatch

The *pycatch* class is the primary access point to pyCATCH's functionalities.

```
class pycatch.pycatch.pycatch(restore=None, **kwargs)
```

Bases: object

A Python library for extracting and analyzing coronal holes from solar EUV images and magnetograms.

2.1.1 Attributes

dir

[str] The directory path. Defaults to the user's home directory if not provided.

save_dir

[str] The save directory path. Defaults to the user's home directory if not provided.

map_file

[str] The map file path.

magnetogram_file

[str] The magnetogram file path.

map

[sunpy.map.Map] The loaded and configured EUV map. This map contains both the 2D data array and metadata associated with the EUV observation.

original_map

[sunpy.map.Map] The original EUV map. This map contains the initially loaded EUV observation before any operations.

magnetogram

[sunpy.map.Map] The loaded and configured magnetogram. This map contains both the 2D data array and metadata associated with the magnetic field data.

point

[list of float] Seed point for coronal hole extraction.

curves

[tuple]

- The threshold range.
- The calculated area curves for the coronal hole.
- The uncertainty in the area curves.

threshold

[float] Coronal hole extraction threshold.

type

[str] Placeholder for type information.

rebin_status

[bool] Whether the map was rebinned.

cutout_status

[bool] Whether the map was cutout.

kernel

[int] Size of the circular kernel for morphological operations.

binmap

[sunpy.map.Map] Single 5-level binary map with the coronal hole extraction, where each level represents a different threshold value.

properties

[dict] A dictionary containing the calculate coronal hole properties.

__version__

[str] Version number of pyCATCH

2.1.2 Parameters

dir

[str, optional] Directory for storing and loading data. Default is the home directory.

save_dir

[str, optional] Directory for storing data. Default is the home directory.

map_file

[str, optional] Filepath to EUV/Intensity map, needs to be loadable with `sunpy.map.Map()`. Default is None.

magnetogram_file

[str, optional] Filepath to magnetogram, needs to be loadable with `sunpy.map.Map()`. Default is None.

load

[str, optional] Loads a previously saved pyCATCH object from the specified path, which overrides any other keywords. Default is None.

2.1.3 Returns

None

2.1.4 Methods

`calculate_curves(verbose=True)`

Calculate area and uncertainty curves as a function of intensity.

Parameters

verbose

[bool, optional] Display warnings. Default is True.

Returns

None

`calculate_properties(mag=False, align=False)`

Calculate the morphological coronal hole properties from the extracted binary map.

Parameters**mag**

[bool, optional] Calculate magnetic properties instead. Default is False.

align

[bool, optional] Call `pycatch.calibration_mag()` to align with binary map. Default is False.

Returns

None

`calibration(**kwargs)`

Calibrate the intensity image.

Parameters

**** kwargs (SDO/AIA) :**

deconvolve

[bool or `numpy.ndarray`, optional] Use PSF deconvolution. Default is None. It takes a custom PSF array as input, if True uses `aiapy.psf.deconvolve`. WARNING: can take about 10 minutes.

register

[bool, optional] Co-register the map. Default is True.

normalize

[bool, optional] Normalize intensity to 1s. Default is True.

degradation

[bool, optional] Correct instrument degradation. Default is True.

alc

[bool, optional] Apply Annulus Limb Correction, Python implementation from Verbeek et al. (2014). Default is True.

cut_limb

[bool, optional] Set off-limb pixel values to NaN. Default is True.

**** kwargs (STEREO/SECCHI) :**

deconvolve

[bool, optional] NOT YET IMPLEMENTED FOR STEREO.

register

[bool, optional] Co-register the map. Default is True.

normalize

[bool, optional] Normalize intensity to 1s. Default is True.

alc

[bool, optional] Apply Annulus Limb Correction, Python implementation from Verbeek et al. (2014). Default is True.

cut_limb

[bool, optional] Set off-limb pixel values to NaN. Default is True.

Returns

None

calibration_mag(**kwargs)

Calibrate the magnetogram.

Parameters

**** kwargs (SDO/HMI)**

[]

rotate

[bool, optional] Rotate the map so that North is up. Default is True.

align

[bool, optional] Align with an AIA map. Default is True.

cut_limb

[bool, optional] Set off-limb pixel values to NaN. Default is True.

Returns

None

cutout(top=(1100, 1100), bot=(-1100, -1100))

Cut a subfield of the map (if a magnetogram is loaded, it will also be cut).

Parameters**top**

[tuple, optional] Coordinates of the top-right corner. Default is (1100, 1100).

bot

[tuple, optional] Coordinates of the bottom-left corner. Default is (-1100, -1100)).

Returns

None

download(time, instr='AIA', wave=193, source='SDO', email=None, mode='JSOC',
series='aia.lev1_euv_12s', **kwargs)

Download an EUV map using either VSO or JSOC via SunPy FIDO.

Downloads the closest image within +/- 15 minutes of the provided time.

Parameters**time**

[tuple, list, str, pandas.Timestamp, pandas.Series, pandas.DatetimeIndex, datetime.datetime, date-time.date, numpy.datetime64, numpy.ndarray, astropy.time.Time] The timestamp of the EUV image to download. Must be parseable by *sunpy.time.parse_time()*.

instr

[str, optional] The instrument name. Default is 'AIA'.

wave

[int, optional] The wavelength in Angstroms. Default is 193.

source

[str, optional] The observational source/spacecraft. Default is 'SDO'.

email

[str, optional] User email required for JSOC queries when *mode*='JSOC'. Default is None.

mode

[str, optional] Download client mode, either 'VSO' or 'JSOC'. Default is 'JSOC'.

series

[str, optional] JSOC series name to query when *mode*='JSOC'. Default is 'aia.lev1_euv_12s'.

****kwargs**

[dict, optional] Additional keyword arguments passed to *sunpy.net.Fido.search*.

Returns

None

Notes

Common provider parameters for non-SDO observations: * EIT: instr='EIT', wave=195, source='SOHO'
 * STEREO-A: instr='SECCHI', wave=195, source='STEREO_A' * STEREO-B: instr='SECCHI',
 wave=195, source='STEREO_B'

```
download_magnetogram(cadence=45, time=None, email=None, mode='JSOC', series='hmi.M_720s',
                     **kwargs)
```

Download a magnetogram closest in time to a given date or an existing EUV image map.

Parameters**cadence**

[int, optional] Temporal resolution / cadence expectation in seconds. Default is 45.

time

[tuple, str, pandas.Timestamp, datetime.datetime, astropy.time.Time, or None, optional] Target time of the magnetogram to download, parsed via *sunpy.time.parse_time()*. If None, attempts to extract the date from *self.map.meta['DATE-OBS']*.

email

[str, optional] Email address required for JSOC exports. Required if *mode*='JSOC'.

mode

[{'JSOC', 'VSO'}, optional] Data service interface to query. Default is 'JSOC'.

series

[str, optional] JSOC series name to query. Default is 'hmi.M_720s'.

****kwargs :**

Additional keyword arguments passed directly to *sunpy.net.Fido.search*.

Returns**None**

Sets *self.magnetogram_file* to the path of the downloaded file.

`extract_ch(kernel=None)`

Extract the coronal hole from the intensity map using the selected threshold and seed point. This function outputs a binary map to `pycatch.binmap`.

Parameters

kernel

[int or None, optional] Size of the circular kernel for morphological operations. Default is None. If None, a kernel size depending on resolution will be used.

Returns

None

`load(file=None, mag=False)`

Load maps.

Parameters

mag

[bool, optional] Flag to load a magnetogram. Default is False.

file

[str, optional] Filepath to load a specific map. If not set, it loads `pycatch.map_file` or `pycatch.magnetogram_file`. Default is None.

Returns

None

`plot_map(boundary=True, uncertainty=True, original=False, combined=True, small=False, cutout=None, grid=False, mag=False, fsize=(10, 10), save=False, sfile=None, overwrite=False, **kwargs)`

Display a coronal hole plot.

`print_properties(file=None, overwrite=False)`

Save properties to a text file.

Parameters

file

[str, optional] Filepath to save the data. Default is `pycatch.dir`.

overwrite

[bool, optional] Flag to overwrite the file if it already exists. Default is False.

Returns

None

`rebin(ndim=(1024, 1024), **kwargs)`

Rebin maps to a new resolution (if a magnetogram is loaded, it will also be resampled).

Parameters

ndim

[tuple, optional] New dimensions of the map. Default is (1024, 1024)).

**** kwargs**

[] Additional keyword arguments passed to `sunpy.map.Map.resample` (see `sunpy` documentation for more information).

Returns

None

`save(file=None, overwrite=False, no_original=True)`

Save a pyCATCH object to a pickle file.

Parameters**file**

[str, optional] Filepath to save the object, default is `pycatch.dir`. Default is None.

overwrite

[bool, optional] Flag to overwrite the file if it already exists. Default is False.

no_original

[bool, optional] Flag to exclude saving the original map to save disk space. Default is True.

Returns

None

`save_map(binary=True, original=False, intensity=False, mag=False, file=None, small=False, overwrite=False)`

Save maps to FITS files.

Parameters**binary**

[bool, optional] Save the binary coronal hole map (*self.binmap*). Default is True.

original

[bool, optional] Save the unprocessed original EUV map (*self.original_map*). Default is False.

intensity

[bool, optional] Save the preprocessed EUV intensity map (*self.map*). Default is False.

mag

[bool, optional] Save the magnetogram map (*self.magnetogram*). Default is False.

file

[str, optional] Custom file path/name to save the FITS file. If None, default file names are generated based on observation metadata. Default is None.

small

[bool, optional] Save a cropped region around the coronal hole. Only applicable when *binary=True*. Default is False.

overwrite

[bool, optional] Flag to overwrite existing files. Default is False.

Returns

None

`select(hint=False, fsize=(10, 10))`

Select a seed point from the intensity map.

Parameters

hint

[bool, optional] If True, highlights possible coronal holes. Default is False.

fsize

[tuple, optional] Set the figure size (in inch). Default is (10, 10).

Returns

None

`set_threshold(threshold, median=True, no_percentage=False)`

Set the coronal hole extraction threshold.

Parameters

threshold

[float, int, list, or np.ndarray] The threshold value.

median

[bool, optional] If True, the input is assumed to be a fraction of the median solar disk intensity. Default is True.

no_percentage

[bool, optional] If True, the input is given as a percentage of the median solar disk intensity. Default is False. This only works in conjunction with `median=True`.

Returns

None

`show(original=False, binmap=False, cutout=None, grid=False, mag=False, fsize=(10, 10), **kwargs)`

Display a coronal hole plot, EUV map, binary detection map, or magnetogram.

Parameters

original

[bool, optional] Show the original uncropped/unprocessed image. Default is False. Overrides `binmap`, `mag`, and default map selection.

binmap

[bool, optional] Show the combined binary coronal hole extraction map. Default is False. Overrides `mag` and default map selection, but is overridden by `original`.

cutout

[list of tuple, optional] Display a bounding-box cutout around the target region. Format: [(xbot, ybot), (xtop, ytop)]. Default is None.

grid

[bool, optional] Display heliographic grid lines on the map. Default is False.

mag

[bool, optional] Show the associated magnetogram map instead of the EUV map or coronal hole plot. Default is False. Overridden by *original* and *binmap*.

fsize

[tuple of (int or float), optional] Figure dimensions (width, height) in inches. Default is (10, 10).

****kwargs :**

Additional keyword arguments passed to *sunpy.map.Map.plot()*.

Returns

None

`suggest_threshold()`

Suggest a coronal hole extraction threshold based on the CATCH statistics (Heinemann et al. 2019).

This function calculates a threshold value using the formula: $TH = 0.29 * Im + 11.53$ [DN]

where Im is the median solar disk intensity in Data Numbers (DN).

Parameters

None

Returns

None

`threshold_from_curves(fsize=(10, 5))`

Select a threshold for coronal hole extraction from calculated area and uncertainty curves as a function of intensity.

Before using this function, you need to calculate the curves using *pycatch.calculate_curves()*.

Parameters**fsize**

[tuple, optional] Set the figure size (in inch). Default is (10, 5).

Returns

None

`threshold_from_hist(fsize=(10, 5))`

Select a threshold for coronal hole extraction from the solar disk intensity histogram.

Parameters**fsize**

[tuple, optional] Set the figure size (in inch). Default is (10, 5).

Returns

None

`wipe_properties()`

Resets all property lists to empty lists.

2.2 pycatch.utils

The *utils* package contains utility submodules.

2.2.1 pycatch.utils.calibration Module

The *calibration* module provides calibration functions.

`pycatch.utils.calibration.annulus_limb_correction(map)`

Apply annulus limb correction to a solar map.

This function performs annulus limb correction on a solar map following the method described in Verbeek et al. (2014). Transferred from IDL to Python 3 by S.G. Heinemann, June 2022

Parameters

map

[`sunpy.map.Map`] The input solar map to which the limb correction will be applied.

Returns

sunpy.map.Map

A solar map with annulus limb correction applied.

`pycatch.utils.calibration.calibrate_aia(map, register=True, pointing=True, respike=True, normalize=True, deconvolve=False, psf_model='SJH', alc=True, degradation=True, cut_limb=True)`

Calibrate and preprocess an AIA (Atmospheric Imaging Assembly) map.

This function performs various calibration and preprocessing steps on an AIA map to prepare it for further analysis.

Parameters

map

[`sunpy.map.Map`] The input AIA map to be calibrated and preprocessed.

register

[`bool`, optional] Whether to perform image registration. Default is `True`.

normalize

[`bool`, optional] Whether to normalize the exposure. Default is `True`.

deconvolve

[`bool` or `None` or `numpy.ndarray`, optional] Whether to perform PSF deconvolution. If set to `True`, deconvolution with `aiapy.psf.deconvolve` is applied. If custom PSF array is given, it uses this array instead. If set to `False`, it is not applied.

alc

[`bool`, optional] Whether to perform annulus limb correction. Default is `True`.

degradation

[`bool`, optional] Whether to correct for instrument degradation. Default is `True`.

cut_limb

[`bool`, optional] Whether to cut the limb of the solar disk. Default is `True`.

Returns

sunpy.map.Map

A calibrated and preprocessed AIA map ready for analysis.

```
pycatch.utils.calibration.calibrate_hmi(map, intensity_map, rotate=False, align=True, cut_limb=True)
```

Calibrate and preprocess an HMI (Helioseismic and Magnetic Imager) map.

This function performs various calibration and preprocessing steps on an HMI map to prepare it for further analysis.

Parameters

map

[sunpy.map.Map] The input HMI map to be calibrated and preprocessed.

intensity_map

[sunpy.map.Map] An intensity map to align the HMI map with.

rotate

[bool, optional] Whether to rotate the HMI map to have north up. Default is False. Overridden by align = True

align

[bool, optional] Whether to align the HMI map with an intensity map. Default is True.

cut_limb

[bool, optional] Set off-limb pixel values to NaN. Default is True.

Returns

sunpy.map.Map

A calibrated and preprocessed HMI map ready for analysis.

```
pycatch.utils.calibration.calibrate_stereo(map, register=True, normalize=True, deconvolve=None, alc=True,  
                                           cut_limb=True)
```

Calibrate and preprocess a STEREO EUV (Solar TERrestrial RELations Observatory) map.

This function performs various calibration and preprocessing steps on a STEREO map to prepare it for further analysis.

Parameters

map

[sunpy.map.Map] The input STEREO map to be calibrated and preprocessed.

register

[bool, optional] Whether to perform map rotation to register the map. Default is True.

normalize

[bool, optional] Whether to normalize the exposure. Default is True.

deconvolve

[bool or None or numpy.ndarray, optional] Whether to perform PSF deconvolution. If set to True, deconvolution with aiapy.psf.deconvolve is applied. If custom PSF array is given, it uses this array instead. If set to False, it is not applied.

alc

[bool, optional] Whether to perform annulus limb correction. Default is True.

cut_limb

[bool, optional] Set off-limb pixel values to NaN. Default is True.

Returns**sunpy.map.Map**

A calibrated and preprocessed STEREO map ready for analysis.

```
pycatch.utils.calibration.convolve_image(img, psf, pad=True, large_psf=False)
```

```
pycatch.utils.calibration.deconvolve_bid(img, psf, iterations=25, tolerance=0.1, mask=None, large_psf=False, pad=True, estimate_background=True, constrain_positive=True)
```

Deconvolve an image with the point spread function.

Perform image deconvolution on an image with the instrument point spread function using the bid algorithm published in Hofmeister et al. (2024), The Basic Iterative Deconvolution: A Fast Instrumental Point-Spread Function Deconvolution Method That Corrects for Light That Is Scattered Out of the Field of View of a Detector, Solar Physics, Volume 299, Issue 6, article id.77 <https://ui.adsabs.harvard.edu/abs/2023arXiv231211784H/abstract>

Parameters**img**

[*numpy.ndarray*] A 2D array representing an image.

psf

[*numpy.ndarray*] The point spread function.

iterations

[*int*] Maximum number of iterations.

tolerance

[*float*] The image deconvolution stops when the maximum change from all pixels between the simulated observed image and the observed image is less than TOLERANCE counts.

mask

[*numpy.ndarray*, optional] Allows selecting an image subregion for which the convolution is done. By that, the algorithm can massively speed up. Can be either a 1D array containing the four elements [left, bottom, right, top], or a 2D array masking the pixels that shall be deconvolved. If the 2D mask is used, the boundaries of the 1D array will be calculated from it, i.e., all pixels in a corresponding rectangular box will be deconvolved. At the moment, the dimensions of mask have to be smaller than half of the dimensions of the image. If you need to deconvolve a larger region, deconvolve the entire image instead.

estimate_background

[*bool*, default=True] If a subregion deconvolution is used, it determines if an incoming scattered light estimate from the surrounding region to the subregion should be applied. This increases the fidelity of the result, but costs some computation time. Generally, it is required for average image intensities and below, but is not required for deconvolving bright image regions.

pad

[*bool*, default=True] If True, increase the size of both the PSF and the image by a factor of two, and pad the PSF and image accordingly with zeros. As this is a Fourier-based method, this breaks the symmetric boundary conditions involved in the Fourier transform.

large_psf

[*bool*, default=False] Usually, the PSF has the same dimension as the image, restricting scattered light to half of the image size. If set to True, the PSF given to the deconvolution has to be double the image size (that allows scattering over the full image range). The image will be padded with zeros to match the size of the full PSF, and deconvolution is done over the full PSF.

constrain_positive

[*bool*, default=True] Constrain the deconvolution to positive result intensities. If True, it mitigates small ringing artifacts. If False, allow negative intensities in the reconstructions. Negative intensities are informative, as they can indicate issues (image calibration artifacts, slightly inaccurate PSF, ringing, etc.).

Returns***sunpy.map.Map***

Deconvolved image map.

```
pycatch.utils.calibration.deconvolve_richardson_lucy(img, psf, iterations=25, pad=True, large_psf=False,
                                                    psf_min=0)
```

Deconvolve an image with the point spread function

Perform image deconvolution on an image with the instrument point spread function using the Richardson-Lucy deconvolution algorithm

Parameters**img**

[‘numpy 2d array’] An image.

psf

[~*numpy.ndarray*] The point spread function.

iterations: int

Number of iterations in the Richardson-Lucy algorithm

pad: True/False

If true, increase the size of both the psf and the image by a factor of two, and pad the psf and image accordingly with zeros. As this is a fourier-based method, this breaks the symmetric boundary conditions involved in the fourier transform.

large_psf: True/False

Usually, the PSF has the same dimension as the image, restricting scattered light to half of the image size. If set to true, the PSF given to the deconvolution has to be double the image size (that allows scattering over the full image range). The image will be padded with zeros to match the size of the full psf, and deconvolution is done over the full psf.

Returns**~*sunpy.map.Map***

Deconvolved image

Comments:

Based on the aiapy.deconvolve method, as described in Cheung, M., 2015, *GPU Technology Conference Silicon Valley*, [GPU-Accelerated Image Processing for NASA’s Solar Dynamics Observatory](#)

```
pycatch.utils.calibration.estimate_scattered_light(img_in, psf_in, bd_box_img, large_psf=False, pad=True)
```

```
pycatch.utils.calibration.get_boundary_boxes(mask, img, psf)
```

```
pycatch.utils.calibration.get_psf_cache_dir() → Path
```

Returns the OS-appropriate user cache directory for pycatch: - Linux: ~/.cache/pycatch/psf/ - macOS: ~/Library/Caches/pycatch/psf/ - Windows: C:\Users<User>\AppData\Local\pycatch\psf

```
pycatch.utils.calibration.get_psf_filepath(filename: str) → Path
```

Locates or downloads a PSF file into the OS-specific user cache directory using its direct Dataverse File ID.

`pycatch.utils.calibration.load_psf(filename: str) → CCDDData`

Fetches (if missing) and loads a PSF file into a CCDDData object. <https://dataverse.harvard.edu/dataset.xhtml?persistentId=doi:10.7910/DVN/DYT4ZL>

Example:

```
psf = load_psf("psf_aia_193.fits")
```

`pycatch.utils.calibration.pad_img_psf(img, psf, large_psf=False, constant_values=0.0)`

`pycatch.utils.calibration.rebin_psf(psf, dimension)`

Rebin the PSF to smaller dimensions

If one wants to deconvolve a rebinned image with the PSF, the PSF has to be rebinned accordingly. However, the definition of the center of the PSF is not in the center of the 2d PSF array, but shifted by definition by half a pixel. This function accounts for this shift for rebinning the PSF.

Parameters

psf

[~numpy.ndarray] The point spread function.

dimensions

[[xdim, ydim]] The new size of the PSF.

Returns

np.ndarray

The the rebinned PSF

`pycatch.utils.calibration.unpad_img_psf(img, psf, large_psf=False)`

2.2.2 pycatch.utils.ch_mapping Module

The *ch_mapping* module provides map operation functions.

`pycatch.utils.ch_mapping.calc_area_curves(map, th, kernel, seed, minval, coreg)`

Wrapper to calculate area curves for a coronal hole in a solar map.

Parameters

map

[sunpy.map.Map] The input solar map containing coronal hole data.

th

[float] The threshold value for coronal hole extraction.

kernel

[int or None] The size of the circular kernel for morphological operations.

seed

[tuple] The seed point coordinates (lon, lat) for coronal hole extraction.

minval

[float] The minimum threshold value to consider for area calculation.

coreg

[sunpy.map.Map] The coregistered solar map for area calculation.

Returns

numpy.ndarray

The area curves for the coronal hole as function of intensity.

`pycatch.utils.ch_mapping.catch_calc(binmaps, binary=True)`

Calculate coronal hole properties from a list of binary maps.

This function calculates various properties of coronal holes, such as area, center of mass, extent, and their uncertainties, from a list of binary maps representing different levels of coronal hole segmentation.

Parameters

binmaps

[list of `sunpy.map.Map`] A list of binary maps representing different levels of coronal hole segmentation.

binary

[bool, optional] Indicates whether the input binary maps are binary (Default is True).

Returns

sunpy.map.Map

A binary map representing the combined coronal hole regions.

float

The mean area of coronal holes.

float

The uncertainty of the mean area of coronal holes.

numpy.ndarray

An array containing the mean center of mass (lon, lat) of coronal holes.

numpy.ndarray

An array containing the uncertainty of the mean center of mass (lon, lat) of coronal holes.

numpy.ndarray

An array containing the mean extent (lon1, lon2, lat1, lat2) of coronal holes.

numpy.ndarray

An array containing the uncertainty of the mean extent (lon1, lon2, lat1, lat2) of coronal holes.

`pycatch.utils.ch_mapping.catch_mag(binmaps, magmap)`

Wrapper to calculate magnetic properties of coronal holes from a list of binary maps and a magnetic field map.

This function calculates various magnetic properties of coronal holes, including signed and unsigned mean magnetic flux density, signed and unsigned magnetic flux, and flux balance, from a list of binary maps representing different levels of coronal hole segmentation and a magnetic field map.

Parameters

binmaps

[list of `sunpy.map.Map`] A list of binary maps representing different levels of coronal hole segmentation.

magmap

[`sunpy.map.Map`] The magnetic field map.

Returns**float**

The mean signed magnetic flux density of coronal holes.

float

The uncertainty of the mean signed magnetic flux density of coronal holes.

float

The mean unsigned magnetic flux density of coronal holes.

float

The uncertainty of the mean unsigned magnetic flux density of coronal holes.

float

The mean signed magnetic flux of coronal holes.

float

The uncertainty of the mean signed magnetic flux of coronal holes.

float

The mean unsigned magnetic flux of coronal holes.

float

The uncertainty of the mean unsigned magnetic flux of coronal holes.

float

The mean flux balance of coronal holes.

float

The uncertainty of the mean flux balance of coronal holes.

`pycatch.utils.ch_mapping.catch_uncertainty(x)`

Calculate the uncertainty of a given data array.

This function computes the uncertainty of a given data array using the maximum absolute deviation from the mean.

Parameters**x**`[numpy.ndarray]` The input data array for which uncertainty is calculated.**Returns****float**

The uncertainty value calculated as the maximum absolute deviation from the mean.

`pycatch.utils.ch_mapping.ch_area(map, coreg=None, binary=False)`

Calculate the area of a coronal hole in a solar map.

Parameters**map**`[sunpy.map.Map]` The input solar map containing coronal hole data.**coreg**`[numpy.ndarray, optional]` An optional curvature correction factor for the map. If not provided, it will be computed internally.

binary

[bool, optional] If True, the map is treated as binary data, where coronal hole pixels have a value of 1. If False, the map is thresholded to binary data. Default is False.

Returns**float**

The calculated area of the coronal hole in 10^{10} km^2 .

`pycatch.utils.ch_mapping.ch_flux(binmap, magmap, coreg=[0])`

Calculate magnetic properties of a coronal hole region from a binary map and a magnetic field map.

This function calculates various magnetic properties of a coronal hole region, including signed mean magnetic flux density, unsigned mean magnetic flux density, signed magnetic flux, and unsigned magnetic flux, based on a binary map representing the coronal hole region and a magnetic field map.

Parameters**binmap**

[sunpy.map.Map] A binary map representing the coronal hole region.

magmap

[sunpy.map.Map] The magnetic field map.

coreg

[list of float, optional] An optional curvature correction factor for the map. If not provided, it will be computed internally.

Returns**float**

The mean signed magnetic flux density of the coronal hole region.

float

The mean unsigned magnetic flux density of the coronal hole region.

float

The signed magnetic flux of the coronal hole region.

float

The unsigned magnetic flux of the coronal hole region.

`pycatch.utils.ch_mapping.combine_binmaps(binmap)`

Add multiple binary maps to one binary map.

Parameters**binmaps**

[sunpy.map.Map] A list of 5-level binary maps containing multiple threshold levels.

Returns**sunpy.map.Map**

A single binary map.

`pycatch.utils.ch_mapping.curve_corr(map)`

Compute a correction factor for curvature in a solar map.

Parameters

map

[sunpy.map.Map] The input solar map for which the correction factor is computed.

Returns

numpy.ndarray

A 2D array representing the correction factor for curvature in the solar map.

`pycatch.utils.ch_mapping.cutout(map, top, bot)`

Create a cutout region from a solar map based on top and bottom coordinates.

This function generates a cutout region from the input solar map (*map*) using specified top and bottom coordinates.

Parameters

map

[sunpy.map.Map] The input solar map from which the cutout region will be created.

top

[tuple] A tuple containing the top-right corner coordinates (x, y) of the cutout region in arcseconds.

bot

[tuple] A tuple containing the bottom-left corner coordinates (x, y) of the cutout region in arcseconds.

Returns

sunpy.map.Map

A cutout region of the input solar map based on the specified coordinates.

`pycatch.utils.ch_mapping.extract_ch(map, thr, kernel, seed)`

Extract a coronal hole from a solar map based on a threshold and seed point.

Parameters

map

[sunpy.map.Map] The input solar map containing coronal hole data.

thr

[float] The threshold value for coronal hole extraction.

kernel

[int or None] The size of the circular kernel for morphological operations. If None, the kernel size is automatically determined based on map resolution.

seed

[tuple] The seed point coordinates (x, y) for coronal hole extraction.

Returns

sunpy.map.Map

A “binary” map containing the extracted coronal hole region.

`pycatch.utils.ch_mapping.from_5binmap(binmap)`

Split a 5-level binary map into multiple binary maps.

Parameters

binmap

[sunpy.map.Map] A 5-level binary map containing multiple threshold levels.

Returns

list of sunpy.map.Map

A list of binary maps, one for each threshold level extracted from the input 5-level binary map.

`pycatch.utils.ch_mapping.get_curves(map, seed, kernel=None, upper_lim=False, cores=8)`

Top-level wrapper to calculate coronal hole area curves for a range of threshold values.

This function calculates coronal hole area curves for a range of threshold values, optionally considering an upper limit. It also computes uncertainty in the area curves. The function uses parallel processing to speed up computation.

Parameters

map

[sunpy.map.Map] The input solar map containing coronal hole data.

seed

[tuple] The seed point coordinates (x, y) for coronal hole extraction.

kernel

[int or None, optional] The size of the circular kernel for morphological operations. Default is None.

upper_lim

[int or False, optional] The upper limit for the threshold range. If False, the limit is calculated based on the median value of the solar map data. Default is False.

cores

[int, optional] The number of CPU cores to use for parallel processing. Default is 8.

Returns

tuple

A tuple containing three arrays: - The threshold range. - The calculated area curves for the coronal hole. - The uncertainty in the area curves.

`pycatch.utils.ch_mapping.get_intensity(binmaps, map)`

Calculate intensity properties of coronal holes from a list of binary maps.

This function calculates the mean and median intensity properties of coronal holes from a list of binary maps representing different levels of coronal hole segmentation.

Parameters

binmaps

[list of sunpy.map.Map] A list of binary maps representing different levels of coronal hole segmentation.

map

[sunpy.map.Map] The original intensity map.

Returns**float**

The mean intensity of coronal holes.

float

The uncertainty of the mean intensity of coronal holes.

float

The median intensity of coronal holes.

float

The uncertainty of the median intensity of coronal holes.

`pycatch.utils.ch_mapping.make_circle(s)`

Generate a binary circular mask.

This function generates a binary circular mask with a specified size, where the circle is centered within the mask.

Parameters**s**

[int] The size of the circular mask (side length).

Returns**numpy.ndarray**

A binary circular mask with the specified size.

`pycatch.utils.ch_mapping.min_picker(seed, data)`

Find the minimum value in a local neighborhood around a seed point in a data array.

This function identifies the minimum value within a 5x5 local neighborhood centered around a seed point (*xs*, *ys*) in the input data array.**Parameters****seed**[tuple] The seed point coordinates (*xs*, *ys*) as a tuple.**data**

[array-like] The input data array in which to search for the minimum value.

Returns**float**

The minimum value within the local neighborhood.

`pycatch.utils.ch_mapping.to_5binmap(binmaps)`

Combine multiple binary maps into a single 5-level binary map.

Parameters**binmaps**[list of `sunpy.map.Map`] A list of binary maps to be combined.

Returns

sunpy.map.Map

A single 5-level binary map where each level represents a different threshold value.

2.2.3 pycatch.utils.extensions Module

The *extensions* module provides additional functions.

`pycatch.utils.extensions.congrid(a, newdims, method='linear', centre=False, minusone=False)`

Resample an array to new dimension sizes using various interpolation methods.

Parameters

a

[array-like] The input array to be resampled.

newdims

[tuple] The new dimensions to which the array should be resampled.

method

[str, optional] The interpolation method to use. Default is 'linear'.

centre

[bool, optional] Whether interpolation points are at the centers of the bins. Default is False.

minusone

[bool, optional] Whether to prevent extrapolation one element beyond the bounds of the input array. Default is False.

Returns

array-like

The resampled array with the specified dimensions and interpolation method applied.

Notes

This function is adapted from IDL's *congrid* routine. Arbitrary resampling of source array to new dimension sizes. Currently only supports maintaining the same number of dimensions. To use 1-D arrays, first promote them to shape (x,1). Uses the same parameters and creates the same co-ordinate lookup points as IDL's *congrid* routine, which apparently originally came from a VAX/VMS routine of the same name.

method: neighbour - closest value from original data nearest and linear - uses n x 1-D interpolations using `scipy.interpolate.interp1d` (see Numerical Recipes for validity of use of n 1-D interpolations) spline - uses `ndimage.map_coordinates`

centre: True - interpolation points are at the centres of the bins False - points are at the front edge of the bin

minusone: For example- `inarray.shape = (i,j)` & new dimensions = (x,y) False - inarray is resampled by factors of $(i/x) * (j/y)$ True - inarray is resampled by $(i-1)/(x-1) * (j-1)/(y-1)$ This prevents extrapolation one element beyond bounds of input array.

`pycatch.utils.extensions.find_nearest(array, value)`

Find the index of the nearest value in an array to a specified value.

Parameters

array

[array-like] The input array in which to find the nearest value.

value

[float] The value to which the nearest element in the array is sought.

Returns

int

The index of the nearest value in the array.

`pycatch.utils.extensions.get_extent(map)`

Calculate the extent of a coronal hole in Helioprojective Cartesian (HPC) coordinates.

Parameters

map

[`sunpy.map.Map`] A SunPy map object for which the extent needs to be calculated.

Returns

tuple

A tuple containing two tuples representing the lower-left and upper-right corners of the map's extent in HPC coordinates. The format of the outer tuple is `((x_min, y_min), (x_max, y_max))`, where: `(x_min, y_min)` represents the HPC coordinates of the lower-left corner. `(x_max, y_max)` represents the HPC coordinates of the upper-right corner.

`pycatch.utils.extensions.init_props()`

Initialize a dictionary of properties and their units for pyCATCH.

Returns

dict

A dictionary where keys are property abbreviations, and values are tuples containing the full property name and its unit.

`pycatch.utils.extensions.median_disk(map)`

Calculate the median value within the solar disk region of a map.

Parameters

map

[`sunpy.map.Map`] The input solar map.

Returns

float

The median value of the data within the solar disk.

`pycatch.utils.extensions.printtxt(file, pdict, names, version)`

Write property data to a text file.

This function writes property data to a text file with a specific format. It includes the version number, property names, and units as headers followed by the property values.

Parameters

file : str The filepath to the output text file. **pdict** : dict A dictionary containing property data, where keys correspond to property abbreviations. **names** : dict A dictionary mapping property abbreviations to tuples containing full property names and units. **version** : str The version number of the pyCATCH software.

Returns

None

2.2.4 pycatch.utils.plot Module

The *plot* module provides plotting functions.

```
class pycatch.utils.plot.SnappingCursor(fig, ax, line, line2=None, names=['y'], xe=None, ye=None)
```

Bases: object

A cross-hair cursor that snaps to the data point of a line, which is closest to the x position of the cursor.

For simplicity, this assumes that x values of the data are sorted.

Parameters

fig

[matplotlib.figure.Figure] The matplotlib figure to which the cursor is attached.

ax

[matplotlib.axes.Axes] The matplotlib axes to which the cursor is attached.

line

[matplotlib.lines.Line2D] The line for which the cursor will snap to data points.

line2

[matplotlib.lines.Line2D, optional] An optional second line for which the cursor can snap to data points.

names

[list of str, optional] A list of names for the y-values displayed in the cursor's tooltip.

xe

[array-like, optional] An array of x-values associated with the data points.

ye

[array-like, optional] An array of y-values associated with the data points.

Attributes

location

[float] The x-value of the currently snapped data point.

Methods

on_mouse_click(event)

Handles mouse clicks to capture the cursor's location.

on_mouse_move(event)

Handles mouse movement to snap the cursor to the nearest data point.

on_draw(event)

Handles drawing events to update the cursor's background.

```
create_new_background()
on_draw(event)
on_mouse_click(event)
on_mouse_move(event)
set_cross_hair_visible(visible)
update_background()
```

`pycatch.utils.plot.get_point_from_map(map, hint, fsize, is_notebook=False, max_points=10)`

Display a solar map and allow the user to interactively select a point on the map.

Parameters

map

[sunpy.map.GenericMap] The solar map to be displayed.

fsize

[tuple of int] The size of the figure (width, height) in inches.

is_notebook

[bool] Is pycatch being run in a jupyter notebook

max_points

[int] Max number of coronal holes/points to be selected

Returns

point

[list of float] A list containing the coordinates (x, y) of the selected point on the solar map.

`pycatch.utils.plot.get_thr_from_curves(map, curves, fsize, is_notebook=False)`

Interactively get a threshold value from a plot of coronal hole area curves.

Parameters

map

[sunpy.map.GenericMap] The solar map for which the threshold will be determined.

curves

[tuple] A tuple containing the data for plotting the curves, where *curves[0]* is the x-axis data, *curves[1]* is the y-axis data for the first curve, and *curves[2]* is the y-axis data for the second curve.

fsize

[tuple of int] The size of the figure (width, height) in inches.

Returns

thr

[float] The threshold value determined interactively from the plot.

`pycatch.utils.plot.get_thr_from_hist(map, fsize, is_notebook=False)`

Interactively get a threshold value from a histogram of solar disk data.

Parameters

- map**
[sunpy.map.GenericMap] The solar map for which the threshold will be determined.
- fsize**
[tuple of int] The size of the figure (width, height) in inches.
- is_notebook**
[bool] Is pycatch being run in a jupyter notebook

Returns

- thr**
[float] The threshold value determined interactively from the histogram.

`pycatch.utils.plot.plot_map(map, bmap, boundary, uncertainty, fsize, save, spath, grid, **kwargs)`
Plot a solar map with optional boundaries and uncertainty overlays.

Parameters

- map**
[sunpy.map.GenericMap] The solar map to be plotted.
- bmap**
[sunpy.map.GenericMap or None] A binary mask map for boundaries or uncertainty, or None if not used.
- boundary**
[bool] Whether to plot boundaries on the map.
- uncertainty**
[bool] Whether to overlay uncertainty information on the map.
- fsize**
[tuple of int] The size of the figure (width, height) in inches.
- save**
[bool] Whether to save the figure to a file.
- spath**
[str] The path to save the figure if *save* is True.
- **kwargs**
Additional keyword arguments to pass to the *sunpy.map.Map.plot* function.

Returns

None

`pycatch.utils.plot.show_map(map, grid=False, fsize=(8, 8), **kwargs)`
Plot a quicklook visualization of a solar map with optional magnetogram contours and grid overlays.

Parameters

- map**
[sunpy.map.GenericMap] The primary solar map to be plotted.
- grid**
[bool, optional] Whether to draw heliographic grid lines on the plot. Default is False.

fsize

[tuple of int or float, optional] The size of the figure (width, height) in inches. Default is (8, 8).

****kwargs**

Additional keyword arguments passed directly to *map.plot()*.

Returns

matplotlib.axes.Axes

The WCS axes object containing the solar map plot.

CHANGELOG

3.1 1.0.0 (October 1, 2026)

- Major update for release
 - Updated compatibility to current versions of Python, SunPy, aiapy, ...
 - Included JSOC download capabilities (now standard)
 - Added a new `.show` command for quicklook images of maps
 - Added advanced AIA PSF convolution by S.J. Hofmeister (new code and new PSF that are automatically downloaded)
 - Changed `.bin2fits` to `.save_map` to allow saving of different maps as FITS files (i.e. prepped input data)
 - Fixed data prep with new Python packages
 - Added progress bars to some steps
 - Added compatibility with Jupyter Notebooks
 - Added the capability to extract and analyze multiple coronal holes at the same time for a given image (now arbitrarily capped at 10)
- Bug fixing
 - fixed various small bugs

3.2 0.2.1 (November 9, 2023)

- Bug fixing
 - fixed a bug where importing `pycatch` would not find the `_version.py` file
 - fixed some path issue in the initialization of the home path
 - fixed what version was output in the `.print_properties()` function
 - fixed some function descriptions
 - fixed a problem where the `.print_properties()` function would not write a file if no magnetic properties were calculated.
 - fixed an issue with where possibly additional windows open when trying to select a seed point
- Minor changes
 - changed the keyword order in the `.load()` routine to make it more intuitive
 - added `aiapy` to the list of required packages

- switched default for `.plot_map()` from small from True to False
- added a pdf version of the documentation (User_Manual.pdf that can be found on the github page)

3.3 0.2.0 (September 2023)

- Initial functional beta release (beta - testing)

3.4 0.1.0 (September 2023)

- Initial alpha build

ACKNOWLEDGING OR CITING PYCATCH

If you use pyCATCH in your scientific work, please cite and acknowledge it in your publications.

4.1 DISCLAIMER

If you use pyCATCH versions < 1.0.0 (unpublished version), please get in contact with Stephan G. Heinemann (stephan.heinemann@hmail.at) before publication.

4.1.1 Citing pyCATCH in Publications

Please add the following line within your methods, conclusion or acknowledgements sections:

This research used version X.Y.Z of the pyCATCH open source software package.

The package citation should be to the [CATCH paper](#), with the disclaimer that the python implementation was used.

```
@ARTICLE{Heinemann2019,
  author = {{Heinemann}, Stephan G. and {Temmer}, Manuela and {Heinemann}, Niko and
  ↳ {Dissauer}, Karin and {Samara}, Evangelia and {Jer{\v{c}}i{\v{c}}}, Veronika and {Hofmeister},
  ↳ Stefan J. and {Veronig}, Astrid M.},
  title = "{Statistical Analysis and Catalog of Non-polar Coronal Holes Covering the SDO-Era Using
  ↳ CATCH}",
  journal = {\solphys},
  keywords = {Coronal holes, Magnetic fields, Photosphere, Solar cycle, Observations, Astrophysics -
  ↳ Solar and Stellar Astrophysics},
  year = 2019,
  month = oct,
  volume = {294},
  number = {10},
  eid = {144},
  pages = {144},
  doi = {10.1007/s11207-019-1539-y},
  archivePrefix = {arXiv},
  eprint = {1907.01990},
  primaryClass = {astro-ph.SR},
  adsurl = {https://ui.adsabs.harvard.edu/abs/2019SoPh..294..144H},
  adsnote = {Provided by the SAO/NASA Astrophysics Data System}
}
```

4.1.2 Additional Citations

If you use specific modules within pyCATCH, please cite the corresponding references below:

For the SJH PSF please reference:

```
@ARTICLE{Hofmeister2025,
  author = {{Hofmeister}, Stefan J. and {Savin}, Daniel W. and {Hahn}, Michael},
  title = "{Revised Point-spread Functions for the Atmospheric Imaging Assembly on board the_
↪Solar Dynamics Observatory}",
  journal = {\apjs},
  keywords = {Solar physics, Deconvolution, Solar instruments, 1476, 1910, 1499, Astrophysics - Solar_
↪and Stellar Astrophysics, Astrophysics - Instrumentation and Methods for Astrophysics},
  year = 2025,
  month = may,
  volume = {278},
  number = {1},
  eid = {8},
  pages = {8},
  doi = {10.3847/1538-4365/adbaed},
archivePrefix = {arXiv},
  eprint = {2410.08967},
  primaryClass = {astro-ph.SR},
  adsurl = {https://ui.adsabs.harvard.edu/abs/2025ApJS..278....8H},
  adsnote = {Provided by the SAO/NASA Astrophysics Data System}
}
```

For the Annulus Limb Correction please reference:

```
@ARTICLE{Verbeeck2014,
  author = {{Verbeeck}, C. and {Delouille}, V. and {Mampaey}, B. and {De Visscher}, R.},
  title = "{The SPoCA-suite: Software for extraction, characterization, and tracking of active regions_
↪and coronal holes on EUV images}",
  journal = {\aap},
  keywords = {techniques: image processing, Sun: corona, Sun: activity, Sun: UV radiation},
  year = 2014,
  month = jan,
  volume = {561},
  eid = {A29},
  pages = {A29},
  doi = {10.1051/0004-6361/201321243},
  adsurl = {https://ui.adsabs.harvard.edu/abs/2014A&A...561A..29V},
  adsnote = {Provided by the SAO/NASA Astrophysics Data System}
}
```

REFERENCES

Heinemann, S.G., Temmer, M., Heinemann, N., Dissauer, K., Samara, E., Jerčić, V., Hofmeister, S.J., Veronig, A.M.: 2019, Statistical analysis and catalog of non-polar coronal holes covering the SDO-era using CATCH. *Solar Phys.* 294, 144.

PYTHON MODULE INDEX

p

- `pycatch.utils.calibration`, [16](#)
- `pycatch.utils.ch_mapping`, [20](#)
- `pycatch.utils.extensions`, [27](#)
- `pycatch.utils.plot`, [29](#)

A

annulus_limb_correction() (in module *pycatch.utils.calibration*), 16

C

calc_area_curves() (in module *pycatch.utils.ch_mapping*), 20

calculate_curves() (*pycatch.pycatch.pycatch* method), 8

calculate_properties() (*pycatch.pycatch.pycatch* method), 9

calibrate_aia() (in module *pycatch.utils.calibration*), 16

calibrate_hmi() (in module *pycatch.utils.calibration*), 17

calibrate_stereo() (in module *pycatch.utils.calibration*), 17

calibration() (*pycatch.pycatch.pycatch* method), 9

calibration_mag() (*pycatch.pycatch.pycatch* method), 10

catch_calc() (in module *pycatch.utils.ch_mapping*), 21

catch_mag() (in module *pycatch.utils.ch_mapping*), 21

catch_uncertainty() (in module *pycatch.utils.ch_mapping*), 22

ch_area() (in module *pycatch.utils.ch_mapping*), 22

ch_flux() (in module *pycatch.utils.ch_mapping*), 23

combine_binmaps() (in module *pycatch.utils.ch_mapping*), 23

congrid() (in module *pycatch.utils.extensions*), 27

convolve_image() (in module *pycatch.utils.calibration*), 18

create_new_background() (*pycatch.utils.plot.SnappingCursor* method), 29

curve_corr() (in module *pycatch.utils.ch_mapping*), 23

cutout() (in module *pycatch.utils.ch_mapping*), 24

cutout() (*pycatch.pycatch.pycatch* method), 10

D

deconvolve_bid() (in module *pycatch.utils.calibration*), 18

deconvolve_richardson_lucy() (in module *pycatch.utils.calibration*), 19

download() (*pycatch.pycatch.pycatch* method), 10

download_magnetogram() (*pycatch.pycatch.pycatch* method), 11

E

estimate_scattered_light() (in module *pycatch.utils.calibration*), 19

extract_ch() (in module *pycatch.utils.ch_mapping*), 24

extract_ch() (*pycatch.pycatch.pycatch* method), 11

F

find_nearest() (in module *pycatch.utils.extensions*), 27

from_5binmap() (in module *pycatch.utils.ch_mapping*), 24

G

get_boundary_boxes() (in module *pycatch.utils.calibration*), 19

get_curves() (in module *pycatch.utils.ch_mapping*), 25

get_extent() (in module *pycatch.utils.extensions*), 28

get_intensity() (in module *pycatch.utils.ch_mapping*), 25

get_point_from_map() (in module *pycatch.utils.plot*), 30

get_psf_cache_dir() (in module *pycatch.utils.calibration*), 19

get_psf_filepath() (in module *pycatch.utils.calibration*), 19

get_thr_from_curves() (in module *pycatch.utils.plot*), 30

get_thr_from_hist() (in module *pycatch.utils.plot*), 30

I

init_props() (in module *pycatch.utils.extensions*), 28

L

load() (*pycatch.pycatch.pycatch* method), 12

load_psf() (in module *pycatch.utils.calibration*), 19

M

make_circle() (in module *pycatch.utils.ch_mapping*), 26

median_disk() (in module *pycatch.utils.extensions*), 28

`min_picker()` (in module `pycatch.utils.ch_mapping`), 26
module
 `pycatch.utils.calibration`, 16
 `pycatch.utils.ch_mapping`, 20
 `pycatch.utils.extensions`, 27
 `pycatch.utils.plot`, 29

O

`on_draw()` (`pycatch.utils.plot.SnappingCursor` method), 30
`on_mouse_click()` (`pycatch.utils.plot.SnappingCursor` method), 30
`on_mouse_move()` (`pycatch.utils.plot.SnappingCursor` method), 30

P

`pad_img_psf()` (in module `pycatch.utils.calibration`), 20
`plot_map()` (in module `pycatch.utils.plot`), 31
`plot_map()` (`pycatch.pycatch.pycatch` method), 12
`print_properties()` (`pycatch.pycatch.pycatch` method), 12
`printtxt()` (in module `pycatch.utils.extensions`), 28
`pycatch` (class in `pycatch.pycatch`), 7
`pycatch.utils.calibration`
 module, 16
`pycatch.utils.ch_mapping`
 module, 20
`pycatch.utils.extensions`
 module, 27
`pycatch.utils.plot`
 module, 29

R

`rebin()` (`pycatch.pycatch.pycatch` method), 12
`rebin_psf()` (in module `pycatch.utils.calibration`), 20

S

`save()` (`pycatch.pycatch.pycatch` method), 13
`save_map()` (`pycatch.pycatch.pycatch` method), 13
`select()` (`pycatch.pycatch.pycatch` method), 14
`set_cross_hair_visible()` (`pycatch.utils.plot.SnappingCursor` method), 30
`set_threshold()` (`pycatch.pycatch.pycatch` method), 14
`show()` (`pycatch.pycatch.pycatch` method), 14
`show_map()` (in module `pycatch.utils.plot`), 31
`SnappingCursor` (class in `pycatch.utils.plot`), 29
`suggest_threshold()` (`pycatch.pycatch.pycatch` method), 15

T

`threshold_from_curves()` (`pycatch.pycatch.pycatch` method), 15

`threshold_from_hist()` (`pycatch.pycatch.pycatch` method), 15
`to_5binmap()` (in module `pycatch.utils.ch_mapping`), 26

U

`unpad_img_psf()` (in module `pycatch.utils.calibration`), 20
`update_background()` (`pycatch.utils.plot.SnappingCursor` method), 30

W

`wipe_properties()` (`pycatch.pycatch.pycatch` method), 15